

Intermediate vs Terminal Operations on Streams

Key Differences

Feature	Intermediate Operations	Terminal Operations
Purpose	Transform/Filter the stream	Produce result or side-effect
Lazy/ Eager	Lazy (not executed immediately)	Eager (executes immediately)
Return Type	Returns a new Stream	Returns non-stream (collection, value, void)
Chaining	Can be chained indefinitely	End the stream pipeline
Execution	Only executed when terminal op is called	Triggers execution of entire pipeline

1. Practical Example with Employee Data

java

```
import java.util.*;
import java.util.stream.*;

class Employee {
    private int id;
    private String name;
    private String department;
    private double salary;

    public Employee(int id, String name, String department, double salary) {
        this.id = id;
        this.name = name;
    }
}
```

```
        this.department = department;
        this.salary = salary;
    }
}
```

```
public int getId() { return id; }
public String getName() { return name; }
public String getDepartment() { return department; }
public double getSalary() { return salary; }
```

```
@Override
public String toString() {
    return String.format("%s (%s) - $%.2f", name, department, salary);
}
}
```

```
public class StreamOperationsDemo {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee(1, "John Doe", "IT", 75000),
            new Employee(2, "Jane Smith", "HR", 68000),
            new Employee(3, "Bob Johnson", "IT", 82000),
            new Employee(4, "Alice Brown", "Finance", 91000),
            new Employee(5, "Charlie Wilson", "Marketing", 62000),
            new Employee(6, "David Lee", "IT", 79000),
            new Employee(7, "Emma Davis", "HR", 72000),
            new Employee(8, "Frank Miller", "Finance", 88000)
        );

        System.out.println("=== INTERMEDIATE OPERATIONS (LAZY) ===\n");

        // INTERMEDIATE: filter() - Filters elements
        Stream<Employee> filteredStream = employees.stream()
            .filter(emp -> emp.getSalary() > 70000);
        System.out.println("1. After filter() - Stream created but NOT executed yet");
        System.out.println("    (No output visible yet)\n");

        // INTERMEDIATE: map() - Transforms elements
        Stream<String> mappedStream = employees.stream()
            .filter(emp -> emp.getSalary() > 70000)
            .map(Employee::getName);
        System.out.println("2. After map() - Still lazy, nothing executed");
    }
}
```

```
System.out.println("    (Still no output)\n");

// INTERMEDIATE: sorted() - Sorts elements
Stream<Employee> sortedStream = employees.stream()
    .filter(emp -> emp.getSalary() > 70000)
    .sorted(Comparator.comparing(Employee::getName));
System.out.println("3. After sorted() - Still lazy");
System.out.println("    (No output yet)\n");

// INTERMEDIATE: distinct() - Removes duplicates
Stream<String> distinctStream = employees.stream()
    .map(Employee::getDepartment)
    .distinct();
System.out.println("4. After distinct() - Still lazy");
System.out.println("    (No output yet)\n");

System.out.println("=== TERMINAL OPERATIONS (EAGER) ===\n");

// TERMINAL: forEach() - Executes and prints
System.out.println("5. forEach() - Terminal operation triggers execution:");
employees.stream()
    .filter(emp -> emp.getSalary() > 70000)
    .map(Employee::getName)
    .forEach(name -> System.out.println("    " + name));
System.out.println();

// TERMINAL: collect() - Collects results
System.out.println("6. collect() - Terminal operation collects to List:");
List<String> highEarnings = employees.stream()
    .filter(emp -> emp.getSalary() > 70000)
    .map(Employee::getName)
    .collect(Collectors.toList());
System.out.println("    High earners: " + highEarnings);
System.out.println();

// TERMINAL: count() - Counts elements
System.out.println("7. count() - Terminal operation returns count:");
long itCount = employees.stream()
    .filter(emp -> emp.getDepartment().equals("IT"))
    .count();
System.out.println("    IT Department count: " + itCount);
```

```

        System.out.println();

        // TERMINAL: anyMatch() - Returns boolean
        System.out.println("8. anyMatch() - Terminal operation returns boolean:");
        boolean hasHighSalary = employees.stream()
            .anyMatch(emp -> emp.getSalary() > 90000);
        System.out.println("    Any employee with salary > 90000? " + hasHighSalary);
        System.out.println();

        // TERMINAL: findFirst() - Returns Optional
        System.out.println("9. findFirst() - Terminal operation returns Optional:");
        Optional<Employee> firstIT = employees.stream()
            .filter(emp -> emp.getDepartment().equals("IT"))
            .findFirst();
        firstIT.ifPresent(emp -> System.out.println("    First IT employee: " +
emp.getName()));
        System.out.println();
    }
}

```

2. Demonstrating Lazy vs Eager Execution

java

```

public class LazyVsEagerDemo {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("John", "Jane", "Bob", "Alice", "Charlie");

        System.out.println("=== LAZY EXECUTION DEMONSTRATION ===\n");

        // This pipeline won't execute until a terminal operation is called
        Stream<String> stream = names.stream()
            .filter(name -> {
                System.out.println("Filtering: " + name);
                return name.startsWith("J");
            })
            .map(name -> {
                System.out.println("Mapping: " + name);
                return name.toUpperCase();
            })
    }
}

```

```

        .sorted((a, b) -> {
            System.out.println("Sorting: " + a + " vs " + b);
            return a.compareTo(b);
        });

    System.out.println("Stream created but NOTHING executed yet!");
    System.out.println("All operations are LAZY - waiting for terminal operation\n");

    System.out.println("Now calling terminal operation forEach():\n");
    stream.forEach(System.out::println);

    System.out.println("\n=== OBSERVATION ===");
    System.out.println("Notice how filter, map, and sorted operations");
    System.out.println("are executed ONLY when forEach() is called!");
}
}

```

Output:

text

```

=== LAZY EXECUTION DEMONSTRATION ===

Stream created but NOTHING executed yet!
All operations are LAZY - waiting for terminal operation

Now calling terminal operation forEach():

Filtering: John
Mapping: John
Filtering: Jane
Mapping: Jane
Filtering: Bob
Filtering: Alice
Filtering: Charlie
Sorting: JANE vs JOHN
Sorting: JANE vs JOHN
JANE
JOHN

=== OBSERVATION ===

```

Notice how filter, map, and sorted operations are executed ONLY when forEach() is called!

3. Common Intermediate Operations with Examples

java

```
public class IntermediateOperationsDemo {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
        List<String> words = Arrays.asList("apple", "banana", "apple", "cherry", "banana",
"date");

        System.out.println("=== INTERMEDIATE OPERATIONS ===\n");

        // 1. filter() - Selects elements based on condition
        System.out.println("1. filter() - Select even numbers:");
        numbers.stream()
            .filter(n -> n % 2 == 0)
            .forEach(n -> System.out.print(n + " "));
        System.out.println("\n");

        // 2. map() - Transforms each element
        System.out.println("2. map() - Double each number:");
        numbers.stream()
            .map(n -> n * 2)
            .forEach(n -> System.out.print(n + " "));
        System.out.println("\n");

        // 3. flatMap() - Flattens nested structures
        System.out.println("3. flatMap() - Flatten nested lists:");
        List<List<Integer>> nested = Arrays.asList(
            Arrays.asList(1, 2),
            Arrays.asList(3, 4),
            Arrays.asList(5, 6)
        );
        nested.stream()
            .flatMap(List::stream)
            .forEach(n -> System.out.print(n + " "));
        System.out.println("\n");
    }
}
```

```
// 4. distinct() - Removes duplicates
System.out.println("4. distinct() - Remove duplicates:");
words.stream()
    .distinct()
    .forEach(word -> System.out.print(word + " "));
System.out.println("\n");

// 5. sorted() - Sorts elements
System.out.println("5. sorted() - Sort numbers:");
numbers.stream()
    .sorted((a, b) -> b - a) // descending
    .forEach(n -> System.out.print(n + " "));
System.out.println("\n");

// 6. limit() - Limits number of elements
System.out.println("6. limit() - First 5 numbers:");
numbers.stream()
    .limit(5)
    .forEach(n -> System.out.print(n + " "));
System.out.println("\n");

// 7. skip() - Skips first n elements
System.out.println("7. skip() - Skip first 5 numbers:");
numbers.stream()
    .skip(5)
    .forEach(n -> System.out.print(n + " "));
System.out.println("\n");

// 8. peek() - For debugging
System.out.println("8. peek() - Debug pipeline:");
long count = numbers.stream()
    .peek(n -> System.out.println("Processing: " + n))
    .filter(n -> n > 5)
    .peek(n -> System.out.println("  Filtered: " + n))
    .count();
System.out.println("  Count of numbers > 5: " + count);
}
}
```

4. Common Terminal Operations with Examples

java

```
public class TerminalOperationsDemo {
    public static void main(String[] args) {
        List<String> words = Arrays.asList("apple", "banana", "cherry", "date", "elderberry");
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);

        System.out.println("=== TERMINAL OPERATIONS ===\n");

        // 1. forEach() - Performs action on each element
        System.out.println("1. forEach() - Print each word:");
        words.stream().forEach(System.out::println);
        System.out.println();

        // 2. collect() - Collects into collection
        System.out.println("2. collect() - Collect to List:");
        List<String> upperCase = words.stream()
            .map(String::toUpperCase)
            .collect(Collectors.toList());
        System.out.println("    " + upperCase);
        System.out.println();

        // 3. count() - Returns count
        System.out.println("3. count() - Count words starting with 'a':");
        long count = words.stream()
            .filter(w -> w.startsWith("a"))
            .count();
        System.out.println("    Count: " + count);
        System.out.println();

        // 4. reduce() - Reduces to single value
        System.out.println("4. reduce() - Sum of numbers:");
        int sum = numbers.stream()
            .reduce(0, (a, b) -> a + b);
        System.out.println("    Sum: " + sum);
        System.out.println();

        // 5. anyMatch() / allMatch() / noneMatch()
        System.out.println("5. Match operations:");
```

```

boolean hasApple = words.stream().anyMatch(w -> w.equals("apple"));
boolean allLong = words.stream().allMatch(w -> w.length() > 3);
boolean noneShort = words.stream().noneMatch(w -> w.length() < 3);
System.out.println("    Contains 'apple': " + hasApple);
System.out.println("    All length > 3: " + allLong);
System.out.println("    None length < 3: " + noneShort);
System.out.println();

// 6. findFirst() / findAny()
System.out.println("6. Find operations:");
Optional<String> first = words.stream()
    .filter(w -> w.startsWith("c"))
    .findFirst();
Optional<String> any = words.stream()
    .filter(w -> w.startsWith("e"))
    .findAny();
first.ifPresent(f -> System.out.println("    First: " + f));
any.ifPresent(a -> System.out.println("    Any: " + a));
System.out.println();

// 7. min() / max()
System.out.println("7. Min/Max operations:");
Optional<Integer> min = numbers.stream().min(Integer::compareTo);
Optional<Integer> max = numbers.stream().max(Integer::compareTo);
System.out.println("    Min: " + min.orElse(0));
System.out.println("    Max: " + max.orElse(0));
System.out.println();

// 8. toArray()
System.out.println("8. toArray() - Convert to array:");
String[] array = words.stream()
    .filter(w -> w.length() > 4)
    .toArray(String[]::new);
System.out.println("    Array length: " + array.length);
System.out.println("    Elements: " + Arrays.toString(array));
}
}

```

5. Complete Pipeline Example

java

```
public class CompletePipelineDemo {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee(1, "John Doe", "IT", 75000),
            new Employee(2, "Jane Smith", "HR", 68000),
            new Employee(3, "Bob Johnson", "IT", 82000),
            new Employee(4, "Alice Brown", "Finance", 91000),
            new Employee(5, "Charlie Wilson", "Marketing", 62000),
            new Employee(6, "David Lee", "IT", 79000),
            new Employee(7, "Emma Davis", "HR", 72000),
            new Employee(8, "Frank Miller", "Finance", 88000)
        );

        System.out.println("=== COMPLETE STREAM PIPELINE ===\n");

        // Intermediate: filter -> sorted -> map -> limit
        // Terminal: collect
        List<String> topEarnings = employees.stream()
            // 1. Filter: Get employees with salary > 70000
            .filter(emp -> {
                System.out.println("Filtering: " + emp.getName());
                return emp.getSalary() > 70000;
            })
            // 2. Sort: Sort by salary descending
            .sorted((e1, e2) -> {
                System.out.println("Sorting: " + e1.getName() + " vs " + e2.getName());
                return Double.compare(e2.getSalary(), e1.getSalary());
            })
            // 3. Map: Get only names
            .map(emp -> {
                System.out.println("Mapping: " + emp.getName());
                return emp.getName();
            })
            // 4. Limit: Get top 3
            .limit(3)
            // 5. Collect: Collect to List
            .collect(Collectors.toList());

        System.out.println("\nTop 3 Earnings (>70K): " + topEarnings);
    }
}
```

```
        System.out.println("\n=== SUMMARY ===");
        System.out.println("filter, sorted, map, limit - Intermediate (Lazy)");
        System.out.println("collect - Terminal (Eager)");
        System.out.println("All operations executed only when collect() is called!");
    }
}
```

Output:

text

```
=== COMPLETE STREAM PIPELINE ===

Filtering: John Doe
Filtering: Jane Smith
Filtering: Bob Johnson
Filtering: Alice Brown
Filtering: Charlie Wilson
Filtering: David Lee
Filtering: Emma Davis
Filtering: Frank Miller
Sorting: Alice Brown vs Bob Johnson
Sorting: Alice Brown vs Frank Miller
Sorting: Alice Brown vs David Lee
Sorting: Alice Brown vs John Doe
Sorting: Frank Miller vs Bob Johnson
Sorting: Frank Miller vs David Lee
Sorting: Frank Miller vs John Doe
Sorting: David Lee vs Bob Johnson
Sorting: David Lee vs John Doe
Sorting: John Doe vs Bob Johnson
Mapping: Alice Brown
Mapping: Frank Miller
Mapping: David Lee

Top 3 Earners (>70K): [Alice Brown, Frank Miller, David Lee]

=== SUMMARY ===
filter, sorted, map, limit - Intermediate (Lazy)
```

collect - Terminal (Eager)

All operations executed only when collect() is called!
